

웹 개발 실무자를 위한 JavaScript 완전 정복

건강보험심사평가원

2025. 10.16 ~ 17



JavaScript 기초



변수

- 메모리 주소에 접근하기 위해 사람이 이해할 수 있는 언어로 지정한 식별자

값

- 프로그램에 의해 조작될 수 있는 대상
- 리터럴 표기법으로 생성

```
// 숫자 리터럴  
10.50  
1001
```

```
// 문자열 리터럴  
'Hello'  
"World"
```

```
// 불리언 리터럴  
true  
false
```

```
// null 리터럴  
null
```

```
// undefined 리터럴  
undefined
```

```
// 객체 리터럴  
{ name: 'Lee', gender: 'male' }
```

```
// 배열 리터럴  
[ 1, 2, 3 ]
```

```
// 정규표현식 리터럴  
/ab+c/
```

```
// 함수 리터럴  
function() {}
```

데이터 타입

원시 타입 (primitive data type)

- number
- string
- boolean
- null
- undefined
- symbol (New in ECMAScript 6)

객체 타입 (Object data type)

- Object

동적 타이핑

- 변수에 할당된 값의 타입에 의해 동적으로 변수의 타입이 결정

```
// Number
var num1 = 1001;
var num2 = 10.50;

// String
var string1 = 'Hello';
var string2 = "World";

// Boolean
var bool = true;

// null
var foo = null;

// undefined
var bar;

// Object
var obj = { name: 'Lee', age: 23 };

// Array
var array = [ 1, 2, 3 ];

// function
var foo = function() {};
```

하나 이상의 표현식을 대상으로 산술, 할당, 비교, 논리, 타입 연산 등을 수행해 하나의 값을 생성

```
// 산술 연산자
var area = 5 * 4; // 20

// 문자열 연결 연산자
var str = 'My name is ' + 'Lee'; // "My name is Lee"

// 할당 연산자
var color = 'red'; // "red"

// 비교 연산자
var foo = 3 > 5; // false

// 논리 연산자
var bar = (5 > 3) && (2 < 4); // true

// 타입 연산자
var type = typeof 'Hi'; // "string"

// 인스턴스 생성 연산자
var today = new Date(); // Sat Dec 01 2018 00:57:19 GMT+0900 (한국 표준시)
```

표현식(expression)

- 하나의 값으로 평가(evaluation)
- 값(리터럴), 변수, 객체의 프로퍼티, 배열의 요소, 함수 호출, 메서드 호출, 피연산자와 연산자의 조합

// 표현식

5 // 5

5 * 10 // 50

5 * 10 > 10 // true

(5 * 10 > 10) && (5 * 10 < 100) // true

암묵적 타입 변환

자바스크립트 엔진은 표현식을 평가할 때 문맥(Context)을 고려해 암묵적 타입 변환을 실행

// 표현식이 모두 문자열 타입이어야 하는 컨텍스트

'10' + 2 // '102'

`1 \* 10 = \${ 1 \* 10 }` // "1 * 10 = 10"

// 표현식이 모두 숫자 타입이어야 하는 컨텍스트

5 * '10' // 50

// 표현식이 불리언 타입이어야 하는 컨텍스트

!0 // true

if (1) { }

Truthy/Falsy 값

- 제어문의 조건식과 같이 불리언 값으로 평가되어야 할 컨텍스트에서 false로 평가되는 Falsy 값

```
if (!false)    console.log(false + ' is falsy value');  
if (!undefined) console.log(undefined + ' is falsy value');  
if (!null)     console.log(null + ' is falsy value');  
if (!0)        console.log(0 + ' is falsy value');  
if (!NaN)      console.log(NaN + ' is falsy value');  
if (!'')       console.log('' + ' is falsy value');
```


명시적 타입 변환

■ 문자열 타입으로 변환

```
// 1. String 생성자 함수를 new 연산자 없이 호출하는 방법
// 숫자 타입 => 문자열 타입
console.log(String(1));           // "1"
console.log(String(NaN));         // "NaN"
console.log(String(Infinity));    // "Infinity"
// 불리언 타입 => 문자열 타입
console.log(String(true));        // "true"
console.log(String(false));       // "false"
```

```
// 2. Object.prototype.toString 메소드를 사용하는 방법
// 숫자 타입 => 문자열 타입
console.log((1).toString());      // "1"
console.log((NaN).toString());    // "NaN"
console.log((Infinity).toString()); // "Infinity"
// 불리언 타입 => 문자열 타입
console.log((true).toString());   // "true"
console.log((false).toString());  // "false"
```

```
// 3. 문자열 연결 연산자를 이용하는 방법
// 숫자 타입 => 문자열 타입
console.log(1 + '');              // "1"
console.log(NaN + '');            // "NaN"
console.log(Infinity + '');       // "Infinity"
// 불리언 타입 => 문자열 타입
console.log(true + '');           // "true"
console.log(false + '');          // "false"
```

명시적 타입 변환

■ 숫자 타입으로 변환

// 1. Number 생성자 함수를 new 연산자 없이 호출하는 방법

// 문자열 타입 => 숫자 타입

```
console.log(Number('0'));    // 0
```

```
console.log(Number('-1'));    // -1
```

```
console.log(Number('10.53')); // 10.53
```

// 불리언 타입 => 숫자 타입

```
console.log(Number(true));    // 1
```

```
console.log(Number(false));   // 0
```

// 2. parseInt, parseFloat 함수를 사용하는 방법

// (문자열만 변환 가능)

// 문자열 타입 => 숫자 타입

```
console.log(parseInt('0'));    // 0
```

```
console.log(parseInt('-1'));    // -1
```

```
console.log(parseFloat('10.53')); // 10.53
```

// 3. + 단항 연결 연산자를 이용하는 방법

// 문자열 타입 => 숫자 타입

```
console.log(+ '0');           // 0
```

```
console.log(+ '-1');          // -1
```

```
console.log(+ '10.53');       // 10.53
```

// 불리언 타입 => 숫자 타입

```
console.log(+ true);           // 1
```

```
console.log(+ false);         // 0
```

// 4. * 산술 연산자를 이용하는 방법

// 문자열 타입 => 숫자 타입

```
console.log('0' * 1);          // 0
```

```
console.log('-1' * 1);         // -1
```

```
console.log('10.53' * 1);      // 10.53
```

// 불리언 타입 => 숫자 타입

```
console.log(true * 1);         // 1
```

```
console.log(false * 1);        // 0
```

명시적 타입 변환

■ 불리언 타입으로 변환

```
// 1. Boolean 생성자 함수를 new 연산자 없이 호출하는 방법
// 문자열 타입 => 불리언 타입
console.log(Boolean('x'));      // true
console.log(Boolean(''));       // false
console.log(Boolean('false'));  // true
// 숫자 타입 => 불리언 타입
console.log(Boolean(0));        // false
console.log(Boolean(1));        // true
console.log(Boolean(NaN));      // false
console.log(Boolean(Infinity)); // true
// null 타입 => 불리언 타입
console.log(Boolean(null));     // false
// undefined 타입 => 불리언 타입
console.log(Boolean(undefined)); // false
// 객체 타입 => 불리언 타입
console.log(Boolean({}));       // true
console.log(Boolean([]));       // true
```

```
// 2. ! 부정 논리 연산자를 두 번 사용하는 방법
// 문자열 타입 => 불리언 타입
console.log(!!'x');             // true
console.log(!!'');              // false
console.log(!!'false');         // true
// 숫자 타입 => 불리언 타입
console.log(!!0);               // false
console.log(!!1);               // true
console.log(!!NaN);             // false
console.log(!!Infinity);        // true
// null 타입 => 불리언 타입
console.log(!!null);            // false
// undefined 타입 => 불리언 타입
console.log(!!undefined);       // false
// 객체 타입 => 불리언 타입
console.log(!!{});              // true
console.log(!![]);              // true
```

단축 평가

단축 평가 표현식	평가 결과
true anything	true
false anything	anything
true && anything	anything
false && anything	false

```
// 논리합(||) 연산자
'Cat' || 'Dog' // 'Cat'
false || 'Dog' // 'Dog'
'Cat' || false // 'Cat'
```

```
// 논리곱(&&) 연산자
'Cat' && 'Dog' // Dog
false && 'Dog' // false
'Cat' && false // false
```

```
// 객체가 null인지 확인하고 프로퍼티를 참조
var elem = null;
console.log(elem.value); // TypeError
console.log(elem && elem.value);
```

```
// 단축 평가를 사용한 매개변수의 기본값 설정
function getStringLength(str) {
  str = str || '';
  return str.length;
}
```

```
getStringLength(); // 0
getStringLength('hi'); // 2
```

```
// 참고: ES6의 매개변수의 기본값 설정
function getStringLength(str = '') {
  return str.length;
}
```

객체(object)

- 자바스크립트는 객체 기반의 스크립트 언어이며 원시 타입(Primitives)을 제외한 나머지 값들(함수, 배열, 정규표현식 등)은 모두 객체
- 자바스크립트의 객체는 키(key)와 값(value)으로 구성된 프로퍼티(Property)들의 집합

객체 생성 방법 1. 객체 리터럴

```
var emptyObject = {};  
console.log(typeof emptyObject); // object
```

```
var person = {  
  name: 'Lee',  
  gender: 'male',  
  sayHello: function () { console.log('Hi! My name is ' + this.name); }  
};
```

```
console.log(typeof person); // object  
console.log(person); // {name: "Lee", gender: "male", sayHello: f}  
person.sayHello(); // Hi! My name is Lee
```

객체(object)

객체 생성 방법 2. Object 생성자 함수

```
// 빈 객체의 생성
var person = new Object();

// 프로퍼티 추가
person.name = 'Lee';
person.gender = 'male';
person.sayHello = function () {
    console.log('Hi! My name is ' + this.name);
};

console.log(typeof person); // object
console.log(person); // {name: "Lee", gender: "male", sayHello: f}

person.sayHello(); // Hi! My name is Lee
```

객체(object)

생성자 함수

```
// 생성자 함수
function Person(name, gender) {
  var married = true;      // private
  this.name = name;        // public
  this.gender = gender;    // public
  this.sayHello = function() { // public
    console.log('Hi! My name is ' + this.name);
  };
}
```

```
// 인스턴스의 생성
var person1 = new Person('Lee', 'male');
var person2 = new Person('Kim', 'female');

console.log('person1: ', typeof person1);
console.log('person2: ', typeof person2);
console.log('person1: ', person1);
console.log('person2: ', person2);

person1.sayHello();
person2.sayHello();

console.log(person1.gender); // 'male'
console.log(person1.married); // undefined
```

객체 프로퍼티

프로퍼티 키

- 일반적으로 문자열(빈 문자열 포함)을 지정
- 문자열이나 symbol 값 이외의 값을 지정하면 암묵적으로 타입이 변환되어 문자열이 됨
- 자바스크립트에서 사용 가능한 유효한 이름인 경우, 따옴표를 생략할 수 있음

프로퍼티 값

- 모든 값과 표현식이 올 수 있으며, 프로퍼티 값이 함수인 경우 메소드라고 함

```
var person = {  
  'first-name': 'Ung-mo',  
  'last-name': 'Lee',  
  gender: 'male',  
  1: 10,  
  function: 1 // OK. 하지만 예약어는 사용하지 말아야 한다.  
};
```

```
console.log(person);
```


객체 프로퍼티

프로퍼티 값 읽기

- 마침표(.) 표기법과 대괄호([]) 표기법이 있음

```
var person = {  
  'first-name': 'Ung-mo',  
  'last-name': 'Lee',  
  gender: 'male',  
  1: 10  
};  
  
console.log(person);
```

```
console.log(person.first-name);    // NaN:  
undefined-undefined  
console.log(person[first-name]);   //  
ReferenceError: first is not defined  
console.log(person['first-name']); // 'Ung-mo'
```

```
console.log(person.gender);        // 'male'  
console.log(person[gender]);        // ReferenceError:  
gender is not defined  
console.log(person['gender']);      // 'male'
```

```
console.log(person['1']);           // 10  
console.log(person[1]);             // 10 : person[1] ->  
person['1']  
console.log(person.1);              // SyntaxError
```

객체 프로퍼티

프로퍼티 값 갱신

- 객체가 소유하고 있는 프로퍼티에 새로운 값을 할당

```
var person = {  
  'first-name': 'Ung-mo',  
  'last-name': 'Lee',  
  gender: 'male',  
};  
  
person['first-name'] = 'Kim';  
console.log(person['first-name'] ); // 'Kim'
```

프로퍼티 동적 생성

- 객체가 소유하고 있지 않은 프로퍼티 키에 값을 할당

```
person.age = 20;  
console.log(person.age); // 20
```

객체 프로퍼티

프로퍼티 삭제

- delete 연산자로 객체의 프로퍼티를 삭제

```
var person = {  
  'first-name': 'Ung-mo',  
  'last-name': 'Lee',  
  gender: 'male',  
};
```

```
delete person.gender;  
console.log(person.gender); // undefined
```

```
delete person;  
console.log(person); // Object {first-name: 'Ung-mo', last-name: 'Lee'}
```

객체 프로퍼티

for-in 문

- 객체(배열 포함)에 포함된 모든 프로퍼티에 대해 루프를 수행

```
var person = {  
  'first-name': 'Ung-mo',  
  'last-name': 'Lee',  
  gender: 'male'  
};
```

```
// prop에 객체의 프로퍼티 이름이 반환된다. 단, 순서는 보장되지 않는다.  
for (var prop in person) { console.log(prop + ': ' + person[prop]); }
```

```
var array = ['one', 'two'];
```

```
// index에 배열의 경우 인덱스가 반환된다  
for (var index in array) { console.log(index + ': ' + array[index]); }
```

객체 프로퍼티

- for-in 문은 객체의 문자열 키를 순회하기 위한 문법이므로, 배열에는 사용하지 않는 것을 권장

```
// 배열 요소들만을 순회하지 않는다.
```

```
var array = ['one', 'two'];
```

```
array.name = 'my array';
```

```
for (var index in array) {  
    console.log(index + ': ' + array[index]);  
}
```

- 단점을 보완하기 위해 ES6에서 for-of 문이 추가됨

```
const array = [1, 2, 3];
```

```
array.name = 'my array';
```

```
for (const value of array) {  
    console.log(value);  
}
```

Pass-by-reference

- object type = 객체 타입 = 참조 타입 = 객체의 모든 연산이 실제값이 아닌 참조값으로 처리됨을 의미
- 원시 타입은 값이 한번 정해지면 변경할 수 없지만(immutable), 객체는 프로퍼티를 변경, 추가, 삭제가 가능하므로 변경 가능(mutable)
- 객체 타입은 동적으로 변화할 수 있으므로 어느 정도의 메모리 공간을 확보해야 하는지 예측할 수 없기 때문에 런타임에 메모리 공간을 확보하고 메모리의 힙 영역(Heap Segment)에 저장

```
// Pass-by-reference
```

```
var foo = {  
  val: 10  
}
```

```
var bar = foo;  
console.log(foo.val, bar.val); // 10 10  
console.log(foo === bar);      // true
```

```
bar.val = 20;  
console.log(foo.val, bar.val); // 20 20  
console.log(foo === bar);      // true
```

```
var foo = { val: 10 };  
var bar = { val: 10 };  

```

```
console.log(foo.val, bar.val); // 10 10  
console.log(foo === bar);      // false
```

```
var baz = bar;
```

```
console.log(baz.val, bar.val); // 10 10  
console.log(baz === bar);      // true
```

Pass-by-value

- 원시 타입은 값(value)으로 전달 = 값이 복사되어 전달
- 값이 한번 정해지면 변경할 수 없음 (immutable)
- 런타임(변수 할당 시점)에 메모리의 스택 영역(Stack Segment)에 고정된 메모리 영역을 점유하고 저장

```
// Pass-by-value
```

```
var a = 1;
```

```
var b = a;
```

```
console.log(a, b);    // 1  1
```

```
console.log(a === b); // true
```

```
a = 10;
```

```
console.log(a, b);    // 1  10
```

```
console.log(a === b); // false
```

불변 데이터 패턴(immutable data pattern)

- 레퍼런스를 참조한 다른 객체에서 객체를 변경하는 경우, 의도하지 않은 객체의 변경이 발생
- 객체를 불변객체로 만들어 프로퍼티의 변경을 방지 => 객체의 방어적 복사를 통해 새로운 객체를 생성한 후 변경

Object.assign(target, ...sources)

- ES6에서 추가된 메서드로, 타킷 객체로 소스 객체의 프로퍼티를 복사

```
// Copy
const obj = { a: 1 };
const copy = Object.assign({}, obj);
console.log(copy); // { a: 1 }
console.log(obj === copy); // false
```

```
// Merge
const o1 = { a: 1 };
const o2 = { b: 2 };
const o3 = { c: 3 };

const merge1 = Object.assign(o1, o2, o3);

console.log(merge1); // { a: 1, b: 2, c: 3 }
console.log(o1);     // { a: 1, b: 2, c: 3 }, 타킷
객체가 변경된다!
```

```
// Merge
const o4 = { a: 1 };
const o5 = { b: 2 };
const o6 = { c: 3 };

const merge2 = Object.assign({}, o4, o5, o6);

console.log(merge2); // { a: 1, b: 2, c: 3 }
console.log(o4);     // { a: 1 }
```


불변 데이터 패턴(immutable data pattern)

- Object.assign은 완전한 deep copy를 지원하지 않으므로, 객체 내부의 객체는 Shallow copy 됨

```
const user1 = {  
  name: 'Lee',  
  address: {  
    city: 'Seoul'  
  }  
};
```

```
const user2 = Object.assign({}, user1);  
console.log(user1 === user2); // false
```

```
user2.name = 'Kim';  
console.log(user1.name); // Lee  
console.log(user2.name); // Kim
```

```
// 객체 내부의 객체(Nested Object)는 Shallow copy된다.  
console.log(user1.address === user2.address); // true
```

```
user1.address.city = 'Busan';  
console.log(user1.address.city); // Busan  
console.log(user2.address.city); // Busan
```

불변 데이터 패턴(immutable data pattern)

Object.freeze()

```
const user1 = {  
  name: 'Lee',  
  address: {  
    city: 'Seoul'  
  }  
};
```

```
// Object.assign은 완전한 deep copy를 지원하지 않는다.  
const user2 = Object.assign({}, user1, {name: 'Kim'});
```

```
console.log(user1.name); // Lee  
console.log(user2.name); // Kim
```

```
Object.freeze(user1);
```

```
user1.name = 'Kim'; // 무시된다!
```

```
console.log(user1); // { name: 'Lee', address: { city:  
'Seoul' } }
```

```
console.log(Object.isFrozen(user1)); // true
```

불변 데이터 패턴(immutable data pattern)

내부 객체까지 변경 불가능하게 만들려면 deep freeze해야 함

```
function deepFreeze(obj) {
  const props = Object.getOwnPropertyNames(obj);

  props.forEach((name) => {
    const prop = obj[name];
    if(typeof prop === 'object' && prop !== null) {
      deepFreeze(prop);
    }
  });
  return Object.freeze(obj);
}

const user = {
  name: 'Lee',
  address: {
    city: 'Seoul'
  }
};

deepFreeze(user);

user.name = 'Kim';           // 무시된다
user.address.city = 'Busan'; // 무시된다

console.log(user); // { name: 'Lee', address: { city: 'Seoul' } }
```

함수 정의

- 함수 선언문

```
function square(number) {  
    return number * number;  
}
```

- 함수 표현식

```
var square = function(number) {  
    return number * number;  
};
```

```
var foo = function multiply(a, b) {  
    return a * b;  
};
```

// 기명 함수 표현식(named function expression)

```
var bar = function(a, b) {  
    return a * b;  
};
```

// 익명 함수 표현식(anonymous function expression)

```
console.log(foo(10, 5)); // 50
```

```
console.log(multiply(10, 5)); // Uncaught ReferenceError: multiply is not defined
```

함수 정의

- Function 생성자 함수

```
new Function(arg1, arg2, ... argN, functionBody)
```

```
var square = new Function('number', 'return number * number');  
console.log(square(10)); // 100
```

함수 호이스팅

- 호이스팅: var 선언문이나 function 선언문 등 모든 선언문이 해당 스코프의 선두로 옮겨진 것처럼 동작하는 특성
- 자바스크립트는 모든 선언문(var, let, const, function, function*, class)이 선언되기 이전에 참조 가능
- 함수 표현식의 경우 함수 호이스팅이 아니라 변수 호이스팅이 발생 → 변수 호이스팅은 변수 생성 및 초기화와 할당이 분리되어 진행. 호이스팅된 변수는 undefined로 초기화되고 실제값의 할당은 할당문에서 이루어짐

// 함수 선언문의 경우

```
var res = square(5);
```

```
function square(number) {  
  return number * number;  
}
```

// 함수 표현식의 경우

```
var res = square(5); // TypeError: square is not a function
```

```
var square = function(number) {  
  return number * number;  
}
```

일급 객체(first-class object)

일급 객체: 생성, 대입, 연산, 인자 또는 반환값으로서의 전달 등 프로그래밍 언어의 기본적인 조작을 제한없이 사용할 수 있는 대상을 의미

1. 무명의 리터럴로 표현이 가능하다.
2. 변수나 자료 구조(객체, 배열)에 저장할 수 있다.
3. 함수의 매개변수에 전달할 수 있다.
4. 반환값으로 사용할 수 있다.

```
// 1. 무명의 리터럴로 표현이 가능하다.  
// 2. 변수나 자료 구조에 저장할 수 있다.  
var increase = function (num) {  
  return ++num;  
};
```

```
var decrease = function (num) {  
  return --num;  
};
```

```
var predicates = { increase, decrease };
```

```
// 3. 함수의 매개변수에 전달할 수 있다.  
// 4. 반환값으로 사용할 수 있다.  
function makeCounter(predicate) {
```

```
  var num = 0;  
  
  return function () {  
    num = predicate(num);  
    return num;  
  };  
}  
  
var increaser = makeCounter(predicates.increase);  
console.log(increaser()); // 1  
console.log(increaser()); // 2  
  
var decreaser = makeCounter(predicates.decrease);  
console.log(decreaser()); // -1  
console.log(decreaser()); // -2
```

Call-by-value

원시 타입 인수는 call-by-value(값에 의한 호출)로 동작

```
function foo(primitive) {  
  primitive += 1;  
  return primitive;  
}
```

```
var x = 0;
```

```
console.log(foo(x)); // 1  
console.log(x);      // 0
```


Call-by-reference

객체형(참조형) 인수는 call-by-reference(참조에 의한 호출)로 동작

```
function changeVal(primitive, obj) {  
    primitive += 100;  
    obj.name = 'Kim';  
    obj.gender = 'female';  
}
```

```
var num = 100;  
var obj = {  
    name: 'Lee',  
    gender: 'male'  
};
```

```
console.log(num); // 100  
console.log(obj); // Object {name: 'Lee', gender: 'male'}
```

```
changeVal(num, obj);
```

```
console.log(num); // 100  
console.log(obj); // Object {name: 'Kim', gender: 'female'}
```

함수 객체의 프로퍼티

함수는 객체이므로, 함수도 프로퍼티를 가질 수 있음

```
function square(number) {  
  return number * number;  
}
```

```
square.x = 10;
```

```
square.y = 20;
```

```
console.log(square.x, square.y);
```

```
console.dir(square);
```

```
▼ f square(number) ⓘ  
  x: 10  
  y: 20  
  length: 1  
  name: "square"  
  ▼ prototype:  
    ► constructor: f square(number)  
    ► [[Prototype]]: Object  
  arguments: null  
  caller: null  
  [[FunctionLocation]]: VM143:1  
  ► [[Prototype]]: f ()  
  ► [[Scopes]]: Scopes[1]
```

함수 객체의 프로퍼티

- arguments
 - 함수 호출 시 전달된 인수(argument)들의 정보를 담고 있는 순회가능한(iterable) 유사 배열 객체(array-like object)
 - 함수 내부에서 지역변수처럼 사용되며, 함수 외부에서는 사용할 수 없음
- caller
 - 자신을 호출한 함수를 의미
- length
 - 함수 정의 시 작성된 매개변수 개수를 의미
- name
 - 함수명으로, 기명함수의 경우 함수명을 값으로 가지고, 익명함수의 경우 빈문자열을 값으로 가짐
- __proto__ 접근자 프로퍼티
 - __proto__는 객체가 상속받는 부모 객체인 프로토타입 객체에 대한 참조로, 모든 객체는 이 프로토타입 객체를 통해 메서드나 속성을 상속받게 됨
- prototype
 - 함수 객체만이 소유하는 프로퍼티로 일반 객체에는 없음

함수의 다양한 형태

즉시 실행 함수

- 함수의 정의와 동시에 실행되는 함수
- 최초 한번만 호출되며 다시 호출할 수 없음
- 이러한 특징을 이용해 최초 한번만 실행이 필요한 초기화 처리 등에 사용

내부 함수

- 함수 내부에 정의된 함수
- 내부 함수는 부모 함수 외부에서는 접근할 수 없음
- 내부 함수는 자신을 포함하고 있는 부모 함수의 변수에 접근할 수 있으나, 부모 함수는 자식 함수(내부 함수)의 변수에 접근할 수 없음

재귀 함수

- 자기 자신을 호출하는 함수

콜백 함수

- 함수를 명시적으로 호출하는 방식이 아니라 특정 이벤트가 발생했을 때 시스템에 의해 호출되는 함수
- 대표적인 예로 이벤트 핸들러 처리가 있음

함수 호출 방식과 this 바인딩

- 자바스크립트의 경우 함수 호출 방식에 따라 this에 바인딩할 객체가 동적으로 결정
- 함수를 선언할 때 this에 바인딩할 객체가 정적으로 결정되는 것이 아니고, 함수를 호출할 때 함수가 어떻게 호출되었는지에 따라 this에 바인딩할 객체가 동적으로 결정
- 함수 호출 - 내부 함수는 일반 함수, 메서드, 콜백 함수 어디에서 선언되었든 관계없이 this는 전역 객체를 바인딩
- 메소드 호출 - 메소드 내부의 this는 해당 메소드를 소유한 객체, 즉 해당 메소드를 호출한 객체에 바인딩
- 생성자 함수 호출
 - 기존 함수에 new 연산자를 붙여서 호출하면 해당 함수는 생성자 함수로 동작
 - 일반 함수를 호출하면 this는 전역 객체에 바인딩되지만, new 연산자와 함께 생성자 함수를 호출하면 this는 생성자 함수가 암묵적으로 생성한 빈 객체에 바인딩됨
- apply/call/bind 호출
 - Function.prototype.apply, Function.prototype.call 메소드 : 자바스크립트 엔진의 암묵적 this 바인딩 이외에 this를 특정 객체에 명시적으로 바인딩하는 방법으로 제공
 - Function.prototype.bind : 함수에 인자로 전달한 this가 바인딩된 새로운 함수를 리턴. Function.prototype.bind는 Function.prototype.apply, Function.prototype.call 메소드와 같이 함수를 실행하지 않기 때문에 명시적으로 함수를 호출할 필요가 있음

클로저(closure)

- 함수를 일급 객체로 취급하는 함수형 프로그래밍 언어(엘랭(Erlang), 스칼라(Scala), 하스켈(Haskell), 리스프(Lisp)...)에서 사용되는 중요한 특성
- 함수와 그 함수가 선언됐을 때의 렉시컬 환경(Lexical environment)과의 조합 [MDN]
- 반환된 내부함수가 자신이 선언됐을 때의 환경(Lexical environment)인 스코프를 기억하여 자신이 선언됐을 때의 환경(스코프) 밖에서 호출되어도 그 환경(스코프)에 접근할 수 있는 함수
- 클로저의 활용
 - 현재 상태를 기억하고 변경된 최신 상태를 유지
 - 전역 변수 사용 억제
 - 정보 은닉

```
function outerFunc() {  
  var x = 10;  
  var innerFunc = function () { console.log(x); };  
  return innerFunc;  
}  
  
/**  
 * 함수 outerFunc를 호출하면 내부 함수 innerFunc가 반환된다.  
 * 그리고 함수 outerFunc의 실행 컨텍스트는 소멸한다.  
 */  
var inner = outerFunc();  
inner(); // 10
```

DOM 제어와 이벤트 처리



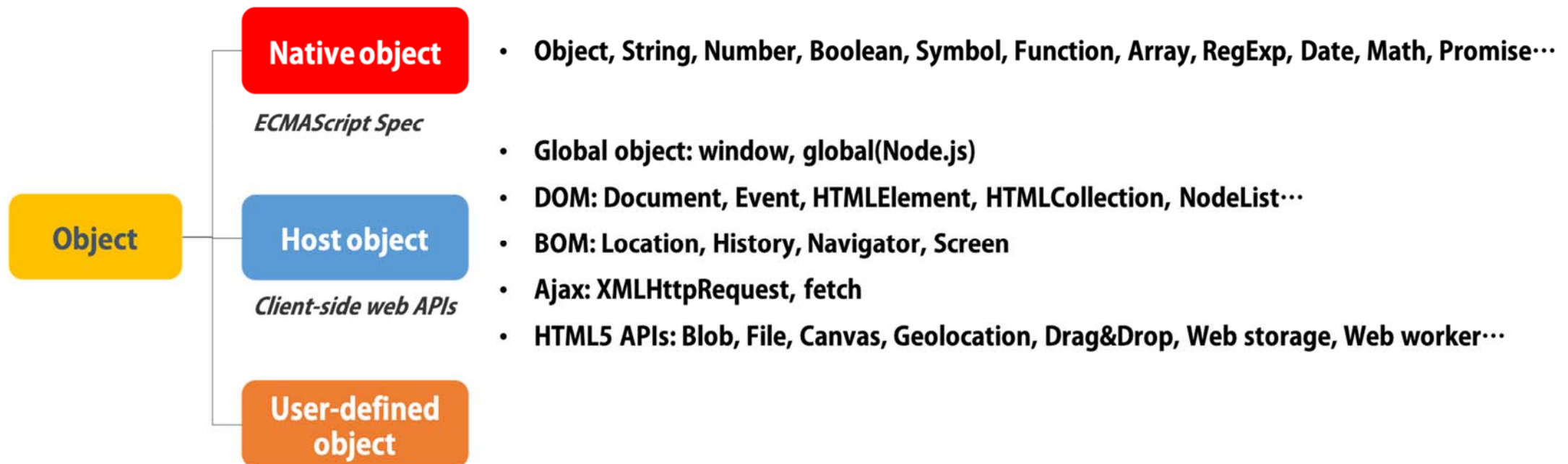
자바스크립트의 객체

■ 네이티브 객체

- ECMAScript 명세에 정의된 객체
- 애플리케이션 전역의 공통 기능을 제공
- 애플리케이션의 환경과 관계없이 언제나 사용 가능
- Object, String, Number, Function, Array, RegExp, Date, Math 등 객체 생성에 관계가 있는 함수 객체와 메소드로 구성

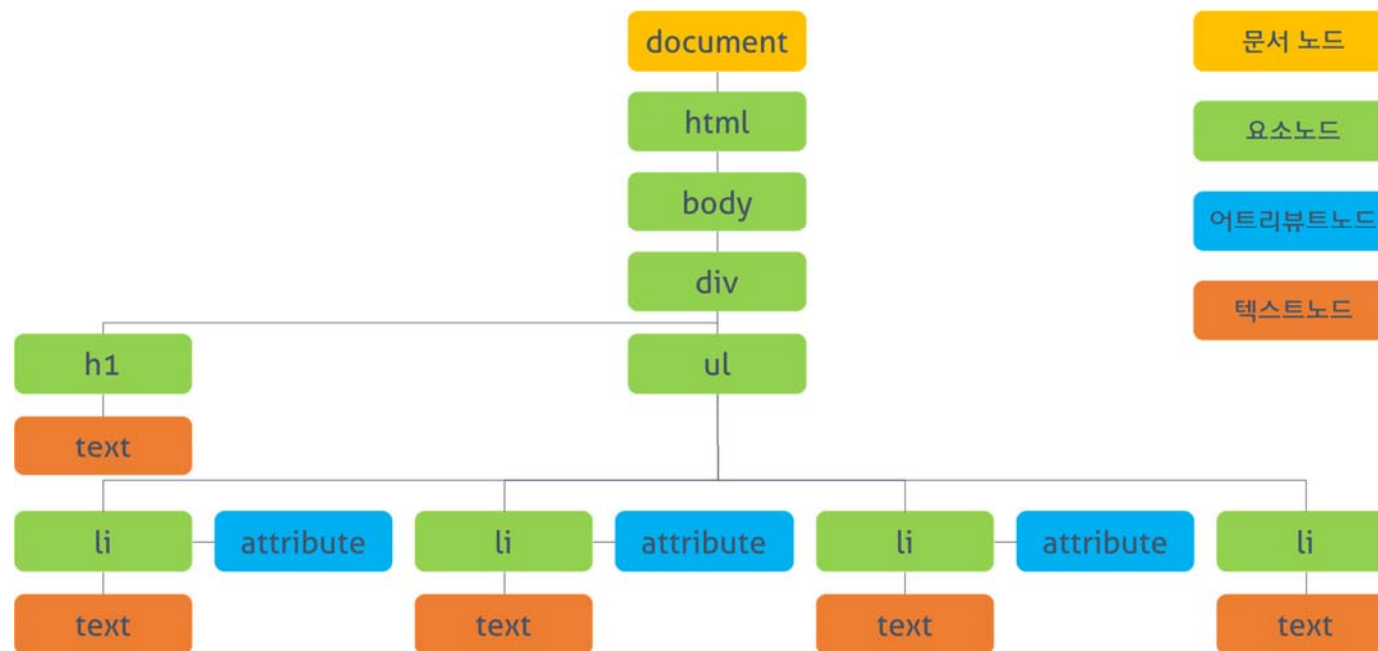
■ 호스트 객체

- 호스트 환경에 정의된 객체
- 브라우저에서 동작하는 환경의 호스트 객체는 전역 객체인 window, BOM(Browser Object Model)과 DOM(Document Object Model), XMLHttpRequest 객체 등을 제공



DOM(Document Object Model)

- 모든 요소와 요소의 어트리뷰트, 텍스트를 각각의 객체로 만들고 이들 객체를 부자 관계를 표현할 수 있는 트리 구조로 구성한 것
- DOM은 자바스크립트를 통해 동적으로 변경할 수 있으며 변경된 DOM은 렌더링에 반영
- DOM은 HTML, ECMAScript에서 정의한 표준이 아닌 별개의 W3C의 공식 표준이며 플랫폼/프로그래밍 언어 종립적으로, HTML 문서에 대한 모델 구성과 HTML 문서 내의 각 요소에 접근, 수정하는 기능을 담당
- DOM tree
 - 브라우저가 HTML 문서를 로드한 후 파싱해 생성하는 모델
 - DOM에서 모든 요소, 어트리뷰트, 텍스트는 하나의 객체이며, Document 객체의 자식이 됨



요소 노드 선택(DOM Query)

- `document.getElementById(id)` - id 어트리뷰트 값으로 요소 노드 한 개를 선택 (복수개가 선택된 경우, 첫번째 요소만 반환)
- `document.querySelector(cssSelector)` - CSS 셀렉터를 사용해 요소 노드 한 개를 선택 (복수개가 선택된 경우, 첫번째 요소만 반환)
- `document.getElementsByClassName(class)` - class 어트리뷰트 값으로 요소 노드를 모두 선택. 공백으로 구분해 여러 개의 class를 지정할 수 있음
- `document.getElementsByTagName(tagName)` - 태그명으로 요소 노드를 모두 선택
- `document.querySelectorAll(selector)` - 지정된 CSS 선택자를 사용해 요소 노드를 모두 선택

DOM Traversing (탐색)

- parentNode - 부모 노드를 탐색
- firstChild, lastChild - 자식 노드를 탐색
- hasChildNodes() - 자식 노드가 있는지 확인하고 Boolean 값을 반환
- childNodes - 자식 노드의 컬렉션을 반환 (텍스트 요소를 포함한 모든 자식 요소를 반환)
- children - 자식 노드의 컬렉션을 반환 (자식 요소 중에서 Element type 요소만을 반환)
- previousSibling, nextSibling - 형제 노드를 탐색 (text node를 포함한 모든 형제 노드를 탐색)
- previousElementSibling, nextElementSibling - 형제 노드를 탐색 (형제 노드 중에서 Element type 요소만을 탐색)

DOM Manipulation (조작)

텍스트 노드에 접근, 수정

1. 해당 텍스트 노드의 부모 노드를 선택한다. 텍스트 노드는 요소 노드의 자식이다.
2. `firstChild` 프로퍼티를 사용하여 텍스트 노드를 탐색한다.
3. 텍스트 노드의 유일한 프로퍼티(`nodeValue`)를 이용하여 텍스트를 취득한다.
4. `nodeValue`를 이용하여 텍스트를 수정한다.

어트리뷰트 노드에 접근, 수정

- `className` - class 어트리뷰트의 값을 취득 또는 변경. class 어트리뷰트의 값이 여러 개일 경우, 공백으로 구분된 문자열을 반환
- `classList` - `add`, `remove`, `item`, `toggle`, `contains`, `replace` 메소드를 제공
- `id` - id 어트리뷰트의 값을 취득 또는 변경
- `hasAttribute(attribute)` - 지정한 어트리뷰트를 가지고 있는지 검사
- `getAttribute(attribute)` - 어트리뷰트의 값을 취득
- `setAttribute(attribute, value)` - 어트리뷰트와 어트리뷰트 값을 설정
- `removeAttribute(attribute)` - 지정한 어트리뷰트를 제거

HTML 콘텐츠 조작

textContent

- 요소의 텍스트 콘텐츠를 취득 또는 변경
- 마크업은 무시
- textContent를 통해 요소에 새로운 텍스트를 할당하면 텍스트 변경 가능. 이때 순수한 텍스트만 지정해야 하며 마크업을 포함시키면 문자열로 인식되어 그대로 출력됨

innerText

- innerText 프로퍼티를 사용하여도 요소의 텍스트 콘텐츠에만 접근 가능
- 비표준이며, CSS에 순종적으로, textContent 프로퍼티 보다 느림 등의 이유로 사용하지 않는 것을 추천

innerHTML

- 해당 요소의 모든 자식 요소를 포함하는 모든 콘텐츠를 하나의 문자열로 취득
- 이 문자열은 마크업을 포함

innerHTML 프로퍼티를 사용하지 않고 새로운 콘텐츠를 추가할 수 있는 방법

DOM 직접 조작

1. 요소 노드 생성 : createElement() 메소드를 사용하여 새로운 요소 노드를 생성한다. createElement() 메소드의 인자로 태그 이름을 전달한다.
2. 텍스트 노드 생성 : createTextNode() 메소드를 사용하여 새로운 텍스트 노드를 생성한다. 경우에 따라 생략될 수 있지만 생략하는 경우, 콘텐츠가 비어 있는 요소가 된다.
3. 생성된 요소를 DOM에 추가 : appendChild() 메소드를 사용하여 생성된 노드를 DOM tree에 추가한다. 또는 removeChild() 메소드를 사용하여 DOM tree에서 노드를 삭제할 수도 있다.

insertAdjacentHTML(position, string)

- 인자로 전달한 텍스트를 HTML로 파싱하고 그 결과로 생성된 노드를 DOM 트리의 지정된 위치에 삽입
- 첫번째 인자는 삽입 위치(beforebegin, afterbegin, beforeend, afterend)
- 두번째 인자는 삽입할 요소를 표현한 문자열

```
<!-- beforebegin -->
<p>
  <!-- afterbegin -->
  foo
  <!-- beforeend -->
</p>
<!-- afterend -->
```

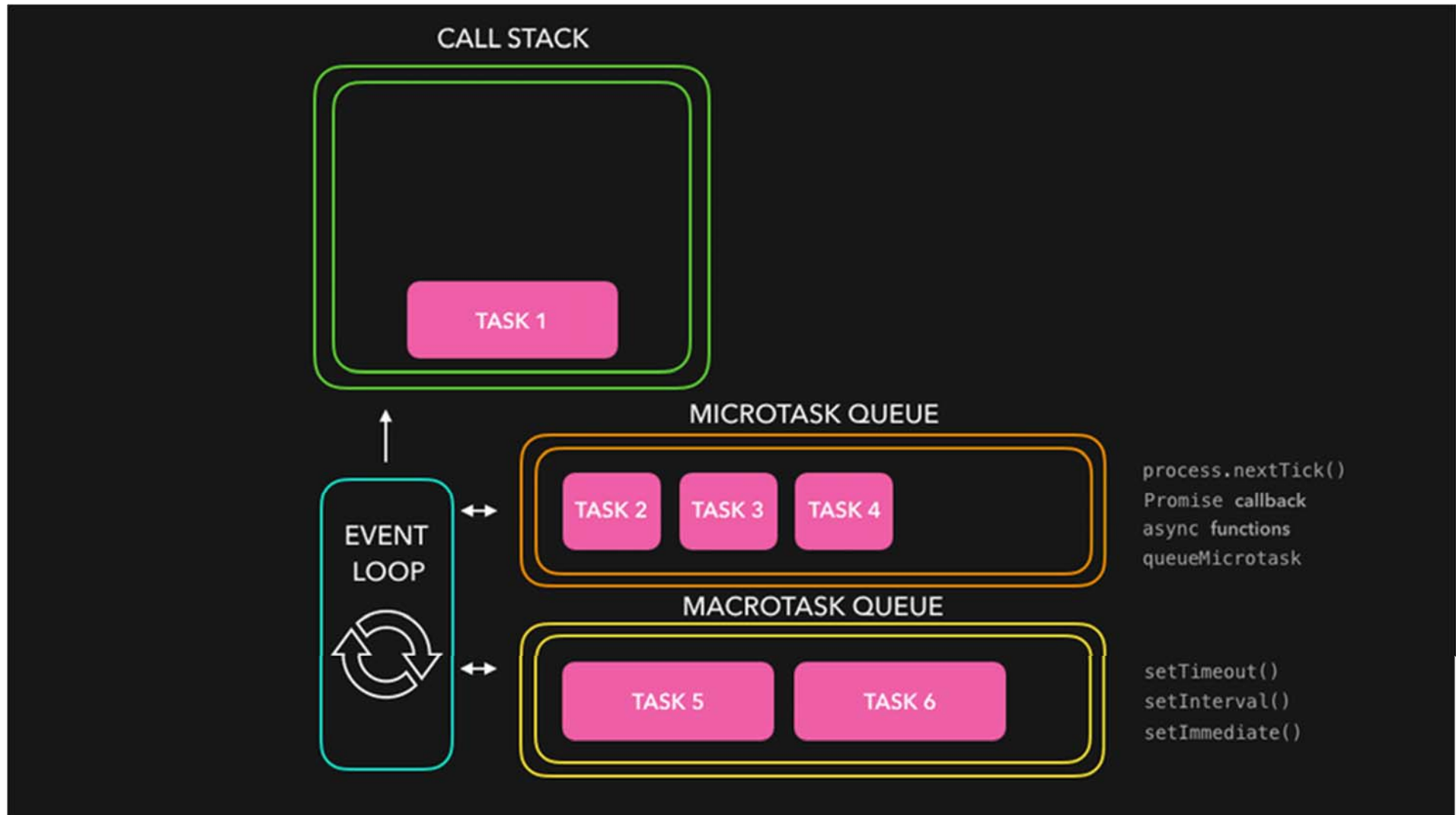
style 프로퍼티

- inline 스타일 선언을 생성해, 특정 요소에 inline 스타일을 지정하는 경우 사용
- style 프로퍼티의 값을 취득하려면 window.getComputedStyle을 사용 - 인자로 주어진 요소의 모든 CSS 프로퍼티 값을 반환

```
/**
 * 요소에 적용된 CSS 프로퍼티를 반환한다.
 * @param {HTMLElement} elem - 대상 요소 노드.
 * @param {string} prop - 대상 CSS 프로퍼티.
 * @returns {string} CSS 프로퍼티의 값.
 */
function getStyle(elem, prop) {
    return window.getComputedStyle(elem, null).getPropertyValue(prop);
}
```

이벤트 루프

이벤트 루프는 콜 스택과 콜백 큐를 지속적으로 확인하며, 콜 스택이 비어있으면 콜백 큐에서 가장 오래된 작업(FIFO)을 꺼내서 콜 스택으로 옮깁니다.



이벤트 종류와 핸들러 등록 방법

이벤트 종류

- https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Events

이벤트 핸들러 등록 방법

- 인라인 이벤트 핸들러 방식
- 이벤트 핸들러 프로퍼티 방식
- `addEventListener` 메소드 방식

EventTarget.addEventListener(*'eventType'*, *funcionName* [, *useCapture*]);

대상요소

대상요소에 바인딩될
이벤트를 나타내는 문자열

이벤트 발생 시에
호출될 함수명
또는 함수 자체

capture 사용 여부
true: capturing / false: Bubbling (Default)

이벤트 핸들러 함수 내부의 this

인라인 이벤트 핸들러 방식

- 인라인 이벤트 핸들러 방식의 경우, 이벤트 핸들러는 일반 함수로서 호출되므로 이벤트 핸들러 내부의 this는 전역 객체 window를 가리킨다.

이벤트 핸들러 프로퍼티 방식

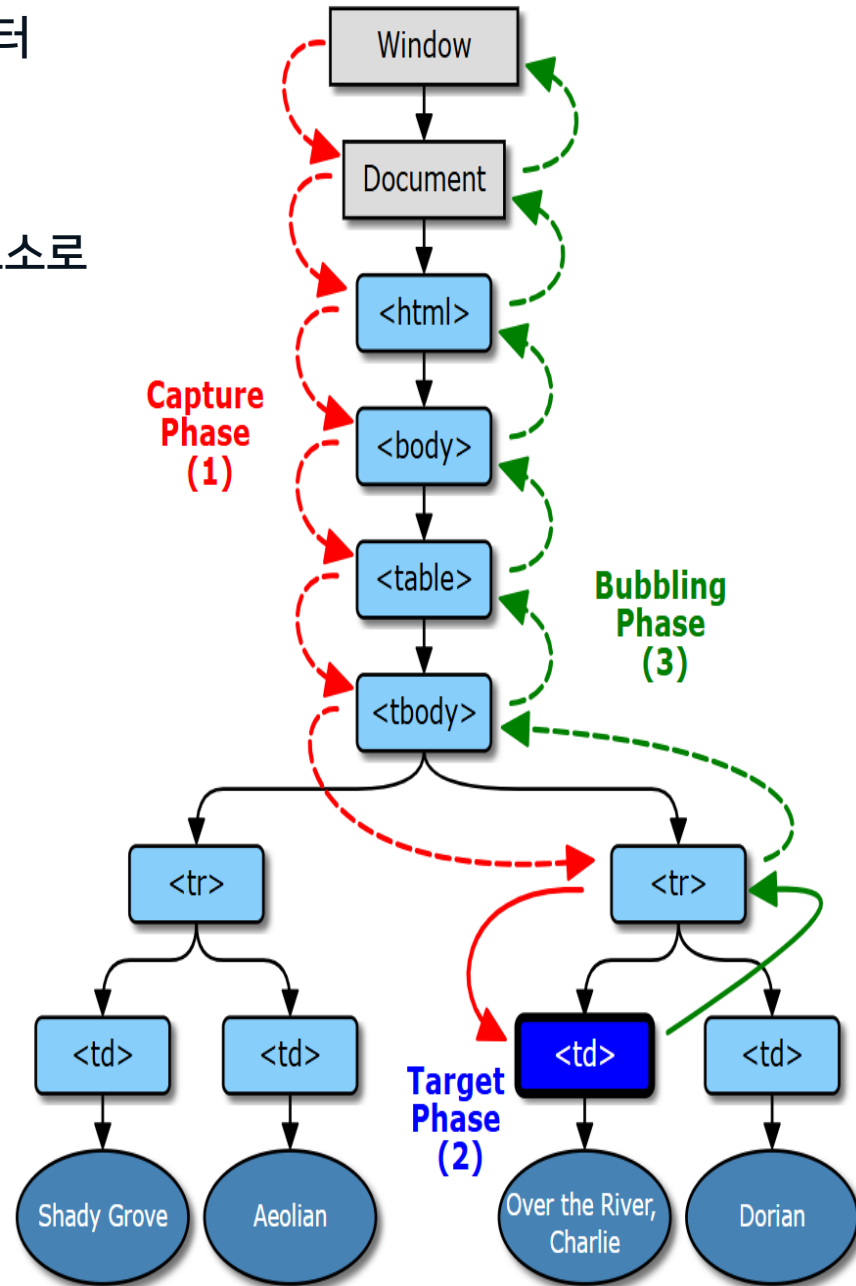
- 이벤트 핸들러 프로퍼티 방식에서 이벤트 핸들러는 메소드이므로 이벤트 핸들러 내부의 this는 이벤트에 바인딩된 요소를 가리킨다. 이것은 이벤트 객체의 currentTarget 프로퍼티와 같다.

addEventListener 메소드 방식

- addEventListener 메소드에서 지정한 이벤트 핸들러는 콜백 함수이지만 이벤트 핸들러 내부의 this는 이벤트 리스너에 바인딩된 요소(currentTarget)를 가리킨다. 이것은 이벤트 객체의 currentTarget 프로퍼티와 같다.

이벤트의 흐름

- 버블링과 캡처링은 둘 중에 하나만 발생하는 것이 아니라 캡처링부터 시작해 버블링으로 종료
- 이벤트가 발생했을 때 캡처링과 버블링은 순차적으로 발생
- 버블링(Event Bubbling): 자식 요소에서 발생한 이벤트가 부모 요소로 전파되는 것
- 캡처링(Event Capturing): 자식 요소에서 발생한 이벤트가 부모 요소부터 시작하여 이벤트를 발생시킨 자식 요소까지 도달하는 것



Event 객체

Event 객체

- 이벤트를 발생시킨 요소와 발생한 이벤트에 대한 유용한 정보를 제공
- 이벤트가 발생하면 event 객체는 동적으로 생성되며 이벤트를 처리할 수 있는 이벤트 핸들러에 인자로 전달

Event Property

- Event.target – 실제로 이벤트를 발생시킨 요소
- Event.currentTarget – 이벤트에 바인딩된 DOM 요소 (addEventListener 앞에 기술된 객체)
- Event.type – 발생한 이벤트의 종류
- Event.cancelable – 요소의 기본 동작을 취소시킬 수 있는지 여부
- Event.eventPhase – 이벤트 흐름 상에서 어느 단계에 있는지를 반환 (0: 이벤트 없음, 1: 캡처링 단계, 2: 타깃, 3: 버블링 단계)

이벤트 위임 및 기본 동작 변경

이벤트 위임(Event Delegation)

- 다수의 자식 요소에 각각 이벤트 핸들러를 바인딩하는 대신 하나의 부모 요소에 이벤트 핸들러를 바인딩하는 방법

기본 동작 변경

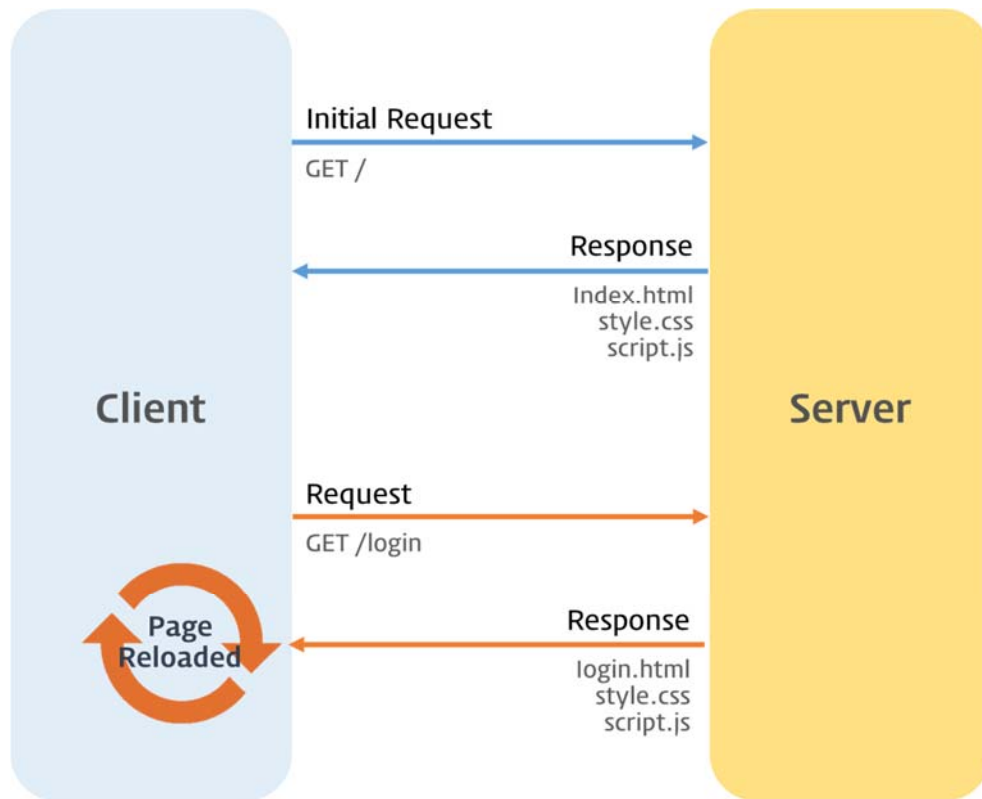
- `Event.preventDefault()` - 폼을 submit하거나 링크를 클릭하면 다른 페이지로 이동하는 것과 같이 요소가 가지고 있는 기본 동작을 위한 메서드
- `Event.stopPropagation()` - 어느 한 요소를 이용해 이벤트를 처리한 후 이벤트가 부모 요소로 이벤트가 전파되는 것을 중단시키기 위한 메소드. 부모 요소에 동일한 이벤트에 대한 다른 핸들러가 지정되어 있을 경우 사용
- `preventDefault & stopPropagation` - 기본 동작의 중단과 버블링 또는 캡처링의 중단을 동시에 실시하는 방법. 단 이방법은 jQuery를 사용할 때만 적용

비동기 처리 및 AJAX

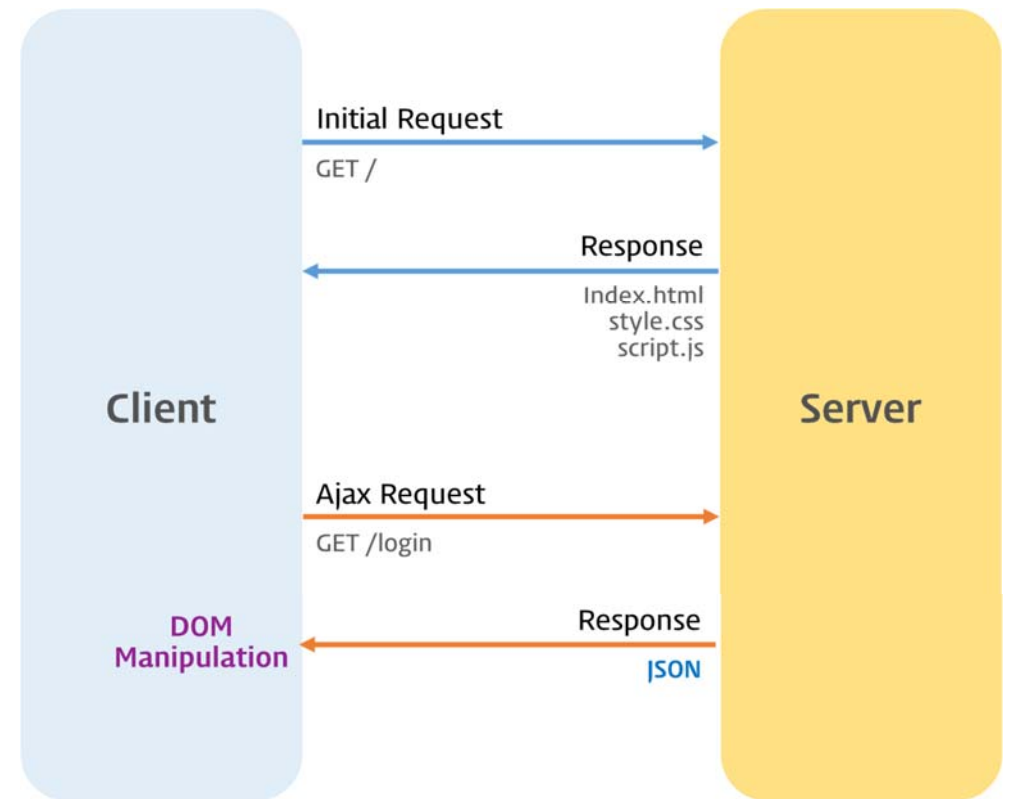


Ajax (Asynchronous JavaScript and XML)

자바스크립트를 이용해서 비동기적(Asynchronous)으로 서버와 브라우저가 데이터를 교환할 수 있는 통신 방식



* Request/Response는 파일 단위로 실시된다. 즉 html, css, js 파일은 한번에 Request/Response되는 것이 아니라 각각의 파일 단위로 Request/Response된다.



* Request/Response는 파일 단위로 실시된다. 즉 html, css, js 파일은 한번에 Request/Response되는 것이 아니라 각각의 파일 단위로 Request/Response된다.

JSON (JavaScript Object Notation)

- 클라이언트와 서버 간 데이터 교환을 위한 규칙 즉 데이터 포맷
- 일반 텍스트 포맷보다 효과적인 데이터 구조화가 가능하며 XML 보다 가볍고 사용하기 간편하며 가독성이 좋음
- 자바스크립트의 객체 리터럴과 매우 흡사
- `JSON.stringify()` - 객체를 JSON 형식의 문자열로 변환
- `JSON.parse()` - JSON 데이터를 가진 문자열을 객체로 변환
- 서버로부터 브라우저로 전송된 JSON 데이터는 문자열이며, 이 문자열을 객체로서 사용하려면 객체화해야 하는데 이를 역직렬화(Deserializing)이라 함. 역직렬화를 위해서 내장 객체 JSON의 static 메소드인 `JSON.parse`를 사용

XMLHttpRequest

- 브라우저는 XMLHttpRequest 객체를 이용해 Ajax 요청을 생성하고 전송
- 서버가 브라우저의 요청에 대해 응답을 반환하면 같은 XMLHttpRequest 객체가 그 결과를 처리

```
const xhr = new XMLHttpRequest(); // XMLHttpRequest 객체의 생성
```

```
xhr.open('GET', '/users'); // 비동기 방식으로 Request를 오픈한다
```

```
xhr.send(); // Request를 전송한다
```

```
// XMLHttpRequest.readyState 프로퍼티가 변경될 때마다 onreadystatechange 이벤트 핸들러가 호출
```

```
xhr.onreadystatechange = function (e) {
```

```
    // readyStates는 XMLHttpRequest의 상태(state)를 반환
```

```
    // readyState: 4 => DONE(서버 응답 완료)
```

```
    if (xhr.readyState !== XMLHttpRequest.DONE) return;
```

```
    // status는 response 상태 코드를 반환 : 200 => 정상 응답
```

```
    if(xhr.status === 200) console.log(xhr.responseText);
```

```
    else console.log('Error!');
```

```
};
```

Fetch API

- 네트워크를 통해 리소스를 비동기적으로 가져오는 데 사용
- AJAX(Asynchronous JavaScript and XML)의 현대적인 대안으로, 주로 서버에 HTTP 요청을 보내고 응답을 받기 위해 사용
- Promise 기반으로 동작하여 콜백 지옥(callback hell)을 피하고 더 깔끔한 코드를 작성할 수 있게 함

```
fetch('https://api.example.com/data')
  .then(response => {
    // 응답이 성공적인지 확인 (HTTP 상태 코드 200-299)
    if (!response.ok) {
      throw new Error('네트워크 응답이 실패했습니다.');
```

- `then()`: `fetch()`가 반환하는 Promise가 성공적으로 이행되었을 때 호출
- 첫 번째 `then()`은 Response 객체를 받으며, 이를 `.json()`이나 `.text()` 등의 메서드를 사용해 응답 본문을 읽기 위한 또 다른 Promise로 변환
- `catch()`: 네트워크 오류나 요청이 실패했을 때 발생한 오류를 처리

Fetch API

- fetch() API는 기본적으로 GET 요청을 보내지만, 두 번째 인자로 옵션 객체를 전달해 다양한 HTTP 메서드(POST, PUT, DELETE 등)를 사용할 수 있음

```
const postData = {  
  name: 'John Doe',  
  age: 30  
};
```

```
fetch('https://api.example.com/users', {  
  method: 'POST', // HTTP 메서드 지정  
  headers: {  
    'Content-Type': 'application/json' // 전송할 데이터의 타입 명시  
  },  
  body: JSON.stringify(postData) // JavaScript 객체를 JSON 문자열로 변환  
})  
  .then(response => response.json())  
  .then(data => console.log('성공:', data))  
  .catch(error => console.error('에러:', error));
```

- method: 요청의 메서드를 정의
- headers: 요청 헤더를 설정
- body: 요청과 함께 전송할 데이터를 포함. 객체는 전송 전에 JSON.stringify()를 사용해 문자열로 변환해야 함

Fetch API

- async/await 구문을 사용하면 fetch() 코드를 더 직관적이고 동기적인 것처럼 작성할 수 있음

```
async function getData() {  
  try {  
    const response = await fetch('https://api.example.com/data');  
    if (!response.ok) {  
      throw new Error('데이터를 가져오는 데 실패했습니다.');    }  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.error('Fetch 에러:', error);  
  }  
}
```

```
getData();
```

- await는 Promise가 이행될 때까지 함수 실행을 일시 중지. 이를 통해 .then() 체인 없이 순차적으로 코드를 작성할 수 있음
- Try...catch 블록을 사용해 오류를 처리

REST(Representational State Transfer)

- HTTP/1.0과 1.1의 스펙 작성에 참여하였고 아파치 HTTP 서버 프로젝트의 공동설립자인 로이 필딩 (Roy Fielding)의 2000년 논문에서 처음 소개
- 발표 당시의 웹이 HTTP의 설계 상 우수성을 제대로 사용하지 못하고 있는 상황을 보고 웹의 장점을 최대한 활용할 수 있는 아키텍처로서 REST를 소개하였고 이는 HTTP 프로토콜을 의도에 맞게 디자인하도록 유도
- REST의 기본 원칙을 성실히 지킨 서비스 디자인을 "RESTful"이라고 표현

REST에서 가장 중요한 기본적인 규칙

1. URI는 정보의 자원을 표현해야 한다.
2. 자원에 대한 행위는 HTTP Method(GET, POST, PUT, DELETE 등)으로 표현한다.

Method	Action	역할	페이로드
GET	index/retrieve	모든/특정 리소스를 조회	x
POST	create	리소스를 생성	○
PUT	replace	리소스의 전체를 교체	○
PATCH	modify	리소스의 일부를 수정	○
DELETE	delete	모든/특정 리소스를 삭제	x

jQuery 기본



- Client-Side Scripting을 단순화하기 위해 설계된 경량의, "write less, do more" 철학을 가진 JavaScript 라이브러리
- 복잡했던 DOM 조작, 이벤트 처리, 애니메이션, Ajax 통신 등을 간결하고 직관적인 문법으로 가능하게 만들어 한때 웹 개발의 표준처럼 사용
- 주요 특징 및 기능
 - HTML/DOM 조작: HTML 문서의 요소를 선택하고, 내용을 변경하며, 속성을 조작하는 작업이 매우 간편. `$()` 함수를 사용해 CSS 선택자와 유사한 방식으로 요소를 쉽게 선택할 수 있음
 - 이벤트 처리: 마우스 클릭, 키보드 입력 등 사용자 상호작용에 대한 이벤트를 간결하게 처리
 - 애니메이션 및 효과: 요소를 부드럽게 나타내거나 사라지게 하는 등의 애니메이션 효과를 손쉽게 구현
 - Ajax: 서버와 비동기적으로 통신하는 Ajax 요청을 간단하게 처리. 페이지를 새로고침하지 않고도 데이터를 주고받는 동적인 웹 페이지를 만드는 데 필수
 - 크로스 브라우징 지원: 과거에는 브라우저마다 다른 JavaScript 문법을 사용해야 했지만, jQuery는 이를 추상화해 모든 주요 브라우저에서 동일하게 작동하도록 보장
- jQuery의 현재 위상 - 예전에 비해 사용이 많이 줄었음
 - 브라우저의 발전: 현대적인 웹 브라우저들은 jQuery가 제공하던 많은 기능을 네이티브 JavaScript API(예: `document.querySelector()`, `addEventListener()`, `fetch()` 등)로 직접 제공 (네이티브 API들은 더 가볍고 빠름)
 - 모던 프레임워크의 등장: React, Vue, Angular와 같은 강력한 JavaScript 프레임워크와 라이브러리가 등장하면서, 개발자들이 DOM을 직접 조작하는 대신 컴포넌트 기반의 추상화된 방식으로 UI를 구축
 - 번들 크기: 경량이라고는 하지만, jQuery 라이브러리 자체의 크기는 페이지 로딩 속도에 영향을 미침. 모던 웹 개발에서는 성능을 최적화하기 위해 필요한 코드만 포함하는 것을 선호

HTML/DOM 조작

요소 선택 (Element Selection)

- `$()` 함수는 CSS 선택자와 동일한 문법을 사용해 하나 이상의 요소를 선택
- `document.querySelector()`나 `document.querySelectorAll()`보다 훨씬 간결

선택자	jQuery 문법	설명
태그	<code>\$("p")</code>	모든 <code><p></code> 요소를 선택합니다.
클래스	<code>\$(".intro")</code>	class가 intro인 모든 요소를 선택합니다.
ID	<code>\$("#header")</code>	id가 header인 단일 요소를 선택합니다.

DOM 탐색 (DOM Traversal)

- 선택된 요소의 부모, 자식, 형제 요소들을 탐색하는 메서드를 제공
- 복잡한 DOM 구조 내에서 원하는 요소를 쉽게 찾을 수 있음
 - `parent()`: 선택된 요소의 직접적인 부모 요소를 반환
 - `children()`: 선택된 요소의 모든 직접적인 자식 요소를 반환
 - `siblings()`: 선택된 요소의 모든 형제 요소를 반환
 - `find()`: 선택된 요소의 하위 요소 중에서 특정 선택자에 해당하는 모든 요소를 찾음
 - `closest()`: 선택된 요소부터 시작해 부모 방향으로 탐색하며, 특정 선택자에 가장 가까운 상위 요소를 찾음

HTML/CSS 조작 (Manipulation)

- 선택한 요소의 콘텐츠(HTML, 텍스트)나 속성(attribute), CSS 스타일을 변경하는 다양한 메서드를 제공
- 콘텐츠 변경
 - `html()`: 요소의 HTML 콘텐츠를 가져오거나 설정
 - `text()`: 요소의 텍스트 콘텐츠를 가져오거나 설정
- 속성 변경
 - `attr()`: 요소의 속성 값을 가져오거나 설정
 - `removeAttr()`: 요소의 속성을 제거
- CSS 스타일 변경
 - `css()`: 요소의 CSS 속성을 가져오거나 설정
 - `addClass()`, `removeClass()`, `toggleClass()`: 클래스를 추가, 제거, 토글

DOM 요소 추가/제거 (Insertion/Deletion)

- 새로운 요소를 생성하거나, 기존 요소를 문서에 추가 또는 제거하는 기능
- 추가
 - append(): 선택된 요소의 끝에 콘텐츠를 추가
 - prepend(): 선택된 요소의 시작에 콘텐츠를 추가
 - after(): 선택된 요소 바로 뒤에 콘텐츠를 추가
 - before(): 선택된 요소 바로 앞에 콘텐츠를 추가
- 제거
 - remove(): 선택된 요소와 그 자식 요소를 모두 제거
 - empty(): 선택된 요소의 자식 요소만 모두 제거

이벤트 처리

이벤트 핸들러 등록

- 가장 기본적인 이벤트 처리 방법은 특정 이벤트가 발생했을 때 실행될 함수(이벤트 핸들러)를 등록하는 것
- 대부분의 표준 HTML 이벤트에 대해 동일한 이름의 메서드를 제공
- 기본 문법: `$(선택자).이벤트명(함수)`

on() 메서드

- 하나의 메서드로 여러 이벤트를 처리하고, 동적으로 생성된 요소에도 이벤트를 바인딩할 수 있는 기능을 제공
- 단일 이벤트 등록 → `$(선택자).on('이벤트명', 함수)`
- 다중 이벤트 등록 → `$(선택자).on({ 이벤트1: 함수1, 이벤트2: 함수2, ... });`
- 동적 요소 이벤트 위임(Delegation)
 - on() 메서드는 이벤트 위임이라는 기법을 사용해 아직 존재하지 않는, 나중에 동적으로 생성될 요소에 대해서도 이벤트를 처리할 수 있게 해줌 → `$(부모_선택자).on('이벤트명', '자식_선택자', 함수)`

이벤트 처리

이벤트 관련 객체 및 메서드

- 이벤트 핸들러 함수 내부에서는 이벤트 객체에 접근할 수 있으며, 이 객체를 통해 이벤트에 대한 자세한 정보를 얻거나 기본 동작을 제어할 수 있음
- event 객체: 이벤트 핸들러의 첫 번째 인자로 전달
 - event.preventDefault(): 브라우저의 기본 동작을 중단
 - event.stopPropagation(): 이벤트 버블링을 중단시켜, 부모 요소로 이벤트가 전파되는 것을 막음
 - \$(this): 이벤트가 발생한 현재 요소

이벤트 버블링과 캡처링

- JavaScript는 이벤트가 발생하면 DOM 트리를 따라 전파되는 이벤트 버블링과 이벤트 캡처링 메커니즘을 가짐
- jQuery는 기본적으로 이벤트 버블링을 따름. 즉, 자식 요소에서 발생한 이벤트가 부모 요소로 전파되며, stopPropagation()을 사용해 이 전파를 막을 수 있음

애니메이션 효과

기본 효과 메서드

- jQuery는 자주 사용되는 애니메이션 효과를 위해 여러 내장 메서드를 제공
- 이 메서드들은 기본적으로 display 속성을 조작
 - show(): 숨겨진 요소를 서서히 나타나게 함
 - hide(): 보이는 요소를 서서히 사라지게 함
 - toggle(): 요소가 보이면 숨기고, 숨겨져 있으면 보이게 전환
 - 이 메서드들은 인자로 지속 시간(slow, fast, 또는 밀리초 단위의 숫자)과 콜백 함수를 받을 수 있으며, 콜백 함수는 애니메이션이 완료된 후 실행됨

페이드(Fade) 효과

- 페이드 효과는 요소의 투명도(opacity)를 조절해 부드럽게 나타나거나 사라지게 함
 - fadeIn(): 요소의 투명도를 0에서 1로 서서히 변경해 나타나게 함
 - fadeOut(): 요소의 투명도를 1에서 0으로 서서히 변경해 사라지게 함
 - fadeToggle(): fadeIn()과 fadeOut()을 전환
 - fadeTo(duration, opacity): 지정된 투명도 값으로 요소를 페이드 인/아웃

애니메이션 효과

슬라이드(Slide) 효과

- 슬라이드 효과는 요소의 높이를 조절해 위/아래로 접히거나 펼쳐지게 함
- 아코디언 메뉴나 토글 가능한 콘텐츠 영역에 자주 사용
 - slideDown(): 높이를 늘려 요소를 아래로 펼침
 - slideUp(): 높이를 줄여 요소를 위로 접음
 - slideToggle(): slideDown()과 slideUp()을 전환

animate() 메서드

- jQuery 애니메이션의 가장 강력한 기능으로, 사용자 정의 CSS 속성을 애니메이션화할 수 있게 함
- \$(선택자).animate({CSS 속성}, 속도, 콜백)
 - {CSS 속성}: 애니메이션을 적용할 CSS 속성을 객체 형태로 지정. 대부분의 숫자 값 속성(예: width, height, left, opacity)에 적용 가능
 - 속도: slow, fast 또는 밀리초로 지정
 - 콜백: 애니메이션이 끝난 후 실행될 함수
- 여러 애니메이션을 큐(queue)에 쌓아 순차적으로 실행할 수 있는 기능도 제공
 - 예) \$("#box").animate({height: '100px'}, 'slow').animate({width: '100px'}, 'slow');

기본 Ajax 메서드

- jQuery는 Ajax를 위한 다양한 간편 메서드를 제공
- `$.ajax()`: 모든 Ajax 작업의 핵심이자 가장 강력한 메서드. 요청 URL, 데이터, 성공/실패 시 콜백 함수 등 모든 설정을 포함하는 객체를 인자로 받음
- `$.get()`: GET 요청을 보내는 약식 메서드로, 데이터를 조회할 때 주로 사용
- `$.post()`: POST 요청을 보내는 약식 메서드로, 서버에 데이터를 생성하거나 업데이트할 때 사용
- `$.getJSON()`: JSON 형식의 데이터를 가져오는 데 특화된 GET 요청 메서드로, 응답을 자동으로 JSON 객체로 파싱

```
$.ajax({  
    url: "요청을 보낼 URL",  
    type: "요청 메서드 (GET, POST 등)",  
    data: "서버로 보낼 데이터",  
    dataType: "서버로부터 받을 데이터 타입 (json, text, html 등)",  
    success: function(data) {  
        // 요청 성공 시 실행될 함수  
    },  
    error: function(jqXHR, textStatus, errorThrown) {  
        // 요청 실패 시 실행될 함수  
    },  
    complete: function(jqXHR, textStatus) {  
        // 요청 완료 시 (성공/실패 무관) 실행될 함수  
    }  
});
```

Ajax 이벤트와 Promises

- jQuery 1.5 버전부터 \$.ajax()는 Deferred Object(Promise와 유사)를 반환
- 이를 통해 .done(), .fail(), .always()와 같은 메서드를 사용해 콜백 함수를 체인 형태로 연결할 수 있음

```
$.ajax({  
  url: "api.example.com/data",  
  type: "GET"  
})  
.done(function(data) { /* 성공 시 */  
  console.log("성공!");  
})  
.fail(function(jqXHR, textStatus, errorThrown) { /* 실패 시 */  
  console.log("실패!");  
})  
.always(function() { /* 항상 */  
  console.log("요청 완료.");  
});
```


ES6(ECMAScript 2015)



let과 const

let

- let은 재할당이 가능한 변수를 선언할 때 사용합니다. 즉, 변수에 새로운 값을 다시 할당할 수 있습니다.
- 블록 스코프: {}로 둘러싸인 코드 블록 내에서만 유효합니다. if, for, while 문 등에서 선언된 let 변수는 해당 블록 밖에서 접근할 수 없습니다.
- 재선언 불가능: 같은 스코프 내에서 동일한 이름의 변수를 다시 선언할 수 없습니다.
- 호이스팅과 TDZ: let 변수는 선언이 코드의 최상단으로 끌어올려지는 호이스팅이 일어나지만, 초기화되지 않은 상태로 존재합니다. 선언 전에 변수에 접근하려 하면 일시적 사각지대(Temporal Dead Zone, TDZ)에 빠져 ReferenceError가 발생합니다.

const

- const는 재할당이 불가능한 상수를 선언할 때 사용합니다. const로 선언된 변수는 한 번 값을 할당하면 변경할 수 없습니다.
- 블록 스코프: let과 마찬가지로 블록 스코프를 가집니다. 재할당 불가능: 선언 시 반드시 값을 할당해야 하며, 이후 값을 변경할 수 없습니다.
- 호이스팅과 TDZ: let과 동일하게 호이스팅되지만, TDZ에 의해 선언 전 접근은 불가능합니다.
- 객체와 배열: const는 변수 자체의 재할당을 막는 것이지, 변수가 가리키는 객체나 배열 내부의 속성까지 변경을 막는 것은 아닙니다. 객체나 배열의 내용은 자유롭게 수정할 수 있습니다.

화살표 함수(Arrow Functions)

화살표 함수(Arrow Functions)는 ES6(ECMAScript 2015)에서 도입된 새로운 함수 표현식으로, 기존의 함수 표현식보다 훨씬 간결하게 함수를 정의할 수 있게 해줍니다.

1. 기본 문법 : 화살표 함수는 `function` 키워드 대신 화살표(`=>`)를 사용해 작성합니다.
2. 매개변수가 하나일 때 : 매개변수가 하나일 때는 소괄호를 생략할 수 있습니다. 매개변수가 없거나 두 개 이상일 때는 소괄호를 반드시 사용해야 합니다.
3. 함수 몸체가 한 줄일 때 : 함수 몸체(body)가 `return` 문 하나로 이루어져 있을 경우, 중괄호 `{}`와 `return` 키워드를 모두 생략할 수 있습니다.

this 바인딩

화살표 함수의 가장 중요한 특징은 `this` 바인딩 방식입니다. 기존 함수와 달리, 화살표 함수는 자신이 정의된 렉시컬 스코프(Lexical Scope)의 `this` 값을 상속받습니다. 이는 동적(Dynamic) 스코프를 가지는 기존 함수와 달리, 화살표 함수가 선언된 정적(Static) 스코프를 따르기 때문입니다.

이러한 특성은 특히 콜백 함수를 사용할 때 유용합니다. 기존 함수에서는 `this` 값을 외부 변수에 할당하거나 `bind()` 메서드를 사용해야 하는 번거로움이 있었지만, 화살표 함수는 자동으로 올바른 `this`를 가리킵니다.

클래스(Classess)

ES6에 새롭게 추가된 클래스 문법은 객체 지향 프로그래밍(OOP)을 더 쉽게 구현할 수 있도록 만들어진 문법적 설탕입니다. 기존 자바스크립트의 프로토타입 기반 상속을 더 익숙한 class 키워드와 extends, constructor 같은 문법으로 사용할 수 있게 해줍니다.

클래스 선언과 생성자

- 클래스는 class 키워드를 사용하여 선언하며, 객체 생성 시 초기화 과정을 담당하는 특별한 메서드인 constructor를 가집니다.
- 클래스 선언 시 constructor는 선택 사항이지만, 인스턴스에 고유한 속성을 부여하려면 반드시 사용해야 합니다.

상속 (extends와 super)

- 클래스는 extends 키워드를 사용해 다른 클래스를 상속받을 수 있습니다. 이를 통해 부모 클래스의 속성과 메서드를 재사용하거나 확장할 수 있습니다. 자식 클래스의 constructor에서는 반드시 super()를 호출해 부모 클래스의 생성자를 먼저 실행해야 합니다.

정적(Static) 메서드

- Static 키워드를 사용하면 클래스 자체에 속하는 정적 메서드를 정의할 수 있습니다. 이 메서드는 클래스 인스턴스가 아닌 클래스 이름으로 직접 호출됩니다. 주로 유틸리티 함수나 인스턴스에 의존하지 않는 작업을 수행할 때 사용합니다.

템플릿 리터럴(Template Literals)

템플릿 리터럴(Template Literals)은 ES6(ECMAScript 2015)에 도입된 문자열 표기법입니다. 기존의 따옴표(" 또는 ") 대신 백틱(`)을 사용해 문자열을 정의합니다. 이를 통해 문자열 내부에 변수를 쉽게 삽입하거나 여러 줄에 걸쳐 문자열을 작성하는 등 다양한 편리한 기능을 제공합니다.

- 변수 삽입 (Interpolation) : 템플릿 리터럴의 가장 큰 장점은 `${...}` 문법을 사용해 문자열 내에 변수나 표현식을 쉽게 삽입할 수 있다는 것입니다. 이를 문자열 보간(String Interpolation)이라고 합니다. 기존의 문자열 연결(+)보다 훨씬 깔끔하고 가독성이 높습니다.
- 여러 줄 문자열 (Multi-line Strings) : 템플릿 리터럴은 줄바꿈을 그대로 반영해 여러 줄의 문자열을 작성할 수 있습니다. 기존에는 줄바꿈을 위해 `\n` 같은 이스케이프 문자를 사용해야 했습니다.
- 표현식 삽입 : 변수뿐만 아니라 함수 호출, 연산 등 유효한 모든 JavaScript 표현식을 `${...}` 안에 포함할 수 있습니다.

템플릿 리터럴의 장점

- 템플릿 리터럴은 단순히 편리함을 넘어 코드의 가독성과 유지보수성을 크게 향상시킵니다. 변수 연결 시 따옴표와 + 기호를 반복해서 사용해야 하는 번거로움을 없애고, 복잡한 문자열을 더 명확하게 표현할 수 있게 해줍니다. 현대 자바스크립트 개발에서는 문자열을 다룰 때 템플릿 리터럴을 사용하는 것이 보편적이고 권장되는 방식입니다.

구조 분해 할당(Destructuring Assignment)

구조 분해 할당(Destructuring Assignment)은 ES6(ECMAScript 2015)에 추가된 문법으로, 배열이나 객체의 속성을 해체해 그 값을 개별 변수에 쉽게 할당할 수 있게 해줍니다.

배열 구조 분해

- 배열의 요소들을 순서대로 개별 변수에 할당할 때 사용합니다. 좌측에는 배열 형태의 변수 목록을, 우측에는 분해할 배열을 배치합니다.
- 일부 요소만 할당하기: 필요한 요소만 선택적으로 가져올 수 있습니다.
- 나머지 요소 분해: 전개 구문(...)을 사용하여 나머지 모든 요소를 새로운 배열로 만들 수 있습니다.

객체 구조 분해

- 객체의 속성들을 개별 변수에 할당할 때 사용합니다. 좌측에는 객체 속성명과 동일한 변수 목록을, 우측에는 분해할 객체를 배치합니다.
- 변수명 변경: {객체_속성: 새로운_변수명} 문법을 사용해 속성을 새로운 이름의 변수에 할당할 수 있습니다.
- 기본값 설정: 존재하지 않는 속성에 대해 기본값을 설정할 수 있습니다.
- 중첩 객체 분해: 중첩된 객체의 속성을 한 번에 분해할 수 있습니다.

모듈(Modules)

모듈은 자바스크립트 코드를 여러 파일로 분리해 관리하고 재사용할 수 있게 해주는 문법입니다.

모듈을 사용하는 이유

- 코드 재사용성: 작성한 함수나 클래스를 다른 파일에서 쉽게 가져와 사용할 수 있습니다.
- 파일 간 의존성 관리: 어떤 파일이 다른 파일에 의존하는지 명확하게 알 수 있습니다.
- 네임스페이스 보호: 모듈 내부의 변수나 함수는 기본적으로 해당 모듈 스코프에 갇히므로, 전역 변수 충돌을 방지합니다.

export

- 다른 파일에서 사용할 수 있도록 변수, 함수, 클래스 등을 내보내는 역할을 합니다.
- 이름을 가진 내보내기 (Named Exports): 여러 요소를 이름으로 내보낼 수 있습니다.
- 기본 내보내기 (Default Exports): 모듈당 하나만 내보낼 수 있으며, 가져올 때 {} 없이 원하는 이름으로 가져올 수 있습니다.

import

- 다른 모듈에서 내보낸 요소를 가져와서 현재 파일에서 사용할 수 있게 합니다.
- 이름으로 가져오기: export된 이름과 동일하게 {}를 사용하여 가져옵니다.
- 기본값 가져오기: {} 없이 원하는 이름으로 가져옵니다.

프라이미스(Promises)

ES6에 추가된 프라이미스(Promise)는 비동기 작업의 최종 완료 또는 실패를 나타내는 객체입니다. 주로 네트워크 요청, 파일 읽기, 타이머 등 시간이 걸리는 작업을 처리할 때 사용되며, 콜백 지옥(Callback Hell)을 해결하기 위한 효과적인 대안으로 등장했습니다.

프라이미스 상태와 작동 방식

- 대기(Pending): 비동기 작업이 아직 완료되지 않은 초기 상태.
- 이행(Fulfilled): 비동기 작업이 성공적으로 완료된 상태.
- 거부(Rejected): 비동기 작업이 실패한 상태.
- 프라이미스는 성공적으로 이행되면 resolve를 호출하고, 실패하면 reject를 호출합니다.

프라이미스 체이닝 (.then(), .catch(), .finally())

- 프라이미스의 가장 큰 장점은 메서드 체이닝을 통해 비동기 코드를 순차적으로 작성할 수 있다는 점입니다.
- .then(): 프라이미스가 이행되었을 때 실행할 콜백 함수를 등록합니다. 성공적으로 전달된 값을 받아서 다음 체인으로 넘길 수 있습니다.
- .catch(): 프라이미스가 거부되었을 때 실행할 콜백 함수를 등록합니다. 에러 처리를 담당합니다.
- .finally(): 프라이미스가 성공하든 실패하든 항상 실행되는 코드를 등록합니다. 로딩 상태 해제 등 마무리 작업에 유용합니다.

기타 중요한 문법

기본 매개변수 (Default Parameters)

- 기본 매개변수는 함수를 호출할 때 인수가 전달되지 않으면 미리 정의된 기본값을 할당하는 기능입니다. 이 기능이 도입되기 전에는 함수 내부에서 `||` 연산자를 사용해 인수의 `undefined` 여부를 확인하는 코드를 작성해야 했습니다.
- 기본 매개변수는 함수의 시그니처 자체를 더 명확하게 만들어 코드를 이해하기 쉽게 합니다.

전개 구문 (Spread Syntax)

- 전개 구문은 `...` (점 세 개) 기호를 사용해 이터러블(iterable) 객체(배열, 문자열 등) 또는 객체의 요소를 개별적인 요소로 전개하는 문법입니다. 주로 배열 합치기, 복사, 함수 인자로 전달할 때 유용하게 쓰입니다.
- 전개 구문은 불변성(Immutability)을 유지하며 데이터를 다룰 때 매우 효과적입니다.

이터레이터와 `for...of` 구문

- 이터레이터(Iterator)는 이터러블(Iterable) 객체(배열, 문자열, 맵, 셋 등)를 순회하기 위한 표준 인터페이스입니다. 이터러블 객체는 `Symbol.iterator`라는 속성을 가지며, 이 속성은 이터레이터 객체를 반환합니다.
- `for...of` 구문은 이 이터레이터 프로토콜을 따르는 객체를 순회하는 새로운 반복문입니다.
- 기존의 `for...in`은 객체의 속성 이름을 순회하지만, `for...of`는 속성 값을 순회합니다.

감사합니다

